
Sistemas Operacionais

Aula 02 - Gestão de Tarefas - PARTE 3

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

7 de abril de 2026

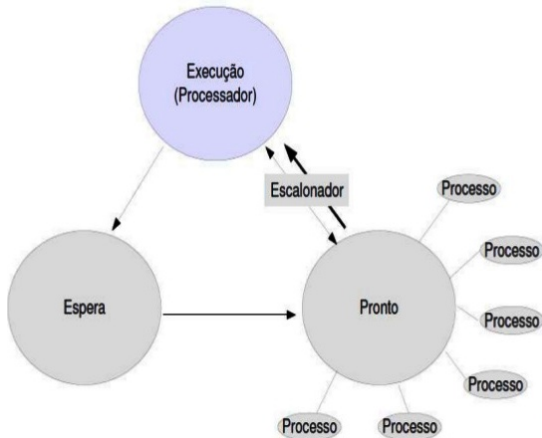


Escalonamento de tarefas



Escalonamento de Tarefas

■ **Tema:** Escalonamento de Processos **Base:** Maziero – Capítulo 6



■ **Objetivo:**

- Entender como o SO decide quem usa a CPU
- Conhecer algoritmos clássicos
- Relacionar teoria com prática em Linux



Atividade pelo qual o sistema operacional decide qual processo ou thread terá o direito de usar a CPU em um determinado momento.

Conhecido também por **scheduling**

Processador é muito mais rápido que os outros componentes

Temos muito mais programas abertos do que núcleos de CPU disponíveis

O escalonador atua como um "juiz" ou "gerente de tráfego"



Por que Escalonamento é Importante?

Imagine um restaurante:

- Vários clientes chegam ao mesmo tempo
- Apenas um cozinheiro (CPU)
- Quem será atendido primeiro?

Escalonador = Gerente da cozinha

- Decide a ordem dos pedidos
- Impacta tempo de espera e satisfação

Analogia:

- Cliente = Processo
- Cozinheiro = CPU
- Fila = Fila de prontos



- **Tempo real:** exigem tempos de resposta precisos.
 - Ex: controle industrial
 - Precisa de resposta previsível
 - deadline obrigatório
- **Interativas:** respondem rapidamente a eventos externos.
 - Ex: navegador, editor, terminal
 - Resposta rápida ao usuário
 - foco: tempo de resposta
- **Batch:** não têm requisitos temporais explícitos.
 - Ex: backup, processamento de dados, renderização
 - Sem urgência
 - foco: throughput



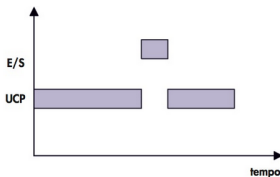
Tipos de tarefas - Em relação ao uso de CPU

■ CPU-bound: tarefas que usam intensivamente a CPU.

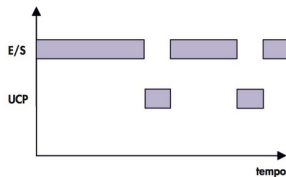
- Usa muito processador
- Ex: renderização, cálculos

■ IO-bound: tarefas que realizam mais entrada/saída.

- Espera por I/O
- Ex: leitura de arquivos, rede



(a) CPU-bound



(b) I/O-bound

■ Analogia:

- CPU-bound = Aluno que senta e resolve exercícios por horas, não para
- IO-bound = Aluno que estuda um pouco, espera resposta do professor / internet, alterna entre estudar e esperar



- Tempo de execução (t_t)
 - intervalo de tempo desde o momento em que o processo é submetido (entra no sistema) até o momento em que ele é completamente finalizado.
- Tempo de espera (t_w)
 - É a soma dos períodos que um processo passou na fila de Pronto (Ready) aguardando a CPU.
- Tempo de resposta (t_r)
 - Minimizar o tempo que o usuário espera após um comando
- Justiça
 - Garantir que nenhum processo "morra de fome"(starvation) esperando para sempre pela CPU.
- Eficiência
 - Finalizar o maior número de tarefas por unidade de tempo.



- Tempo total que um processo leva desde a chegada até sua conclusão.

- **Fórmula**

$$t_t = t_{fim} - t_{chegada}$$

O que inclui?

- Tempo executando (CPU)
 - Tempo esperando (fila)
 - Tempo bloqueado (se houver)
- **Interpretação:** Tempo total que o processo permanece no sistema.
 - **Analogia:** Paciente em hospital: da entrada até a alta.



- Tempo que o processo permanece esperando na fila de pronto

- **Fórmula**

$$t_w = t_t - \text{duração}$$

- **Relação importante**

$$t_t = t_w + \text{tempo de execução}$$

- **Exemplo**

Processo	Chegada	Duração	Fim	T_t	T_w
P1	0	5	5	5	0
P2	1	2	7	6	4

- **Insight**

- T_w alto $\rightarrow$ sistema injusto
- T_t alto $\rightarrow$ sistema lento



Escalonamento cooperativo

- A tarefa só perde o processador ao terminar, solicitar uma entrada/saída ou liberar explicitamente a CPU
- Isto é, a tarefa detém a CPU até que ele mesmo decida liberá-la ou termine sua execução.

Escalonamento preemptivo

- A cada interrupção, exceção ou chamada de sistema, o escalonador reavalia a fila de prontas e pode “preemptar” a tarefa em execução.
- O Sistema Operacional tem o poder de “interromper” um processo à força.
- Se um programa está rodando há muito tempo, o SO suspende ele, salva o que ele estava fazendo e dá a vez para outro.



- Vamos analisar algoritmos mais simples de escalonamento
- Tarefas a escalonar

Tarefa	t_1	t_2	t_3	t_4	t_5
Ingresso	0	0	1	3	5
Duração	5	2	4	1	2
Prioridade	2	3	1	4	5

- OBS: As tarefas não param para realizar operações de E/S.
- Métricas que vamos considerar
 - Tempo médio de execução (T_t) das tarefas
 - Tempo médio de espera (T_w) das tarefas



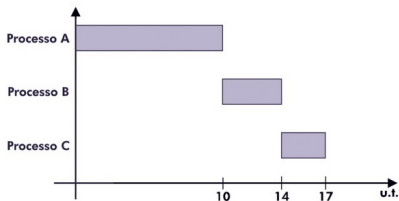
First-Come, First Served (FCFS)

- Também conhecido como fila FIFO (First-In, First-Out)
 - O primeiro que chega é o primeiro a ser servido.
- Fila de prontos (ready queue): os processos são inseridos na ordem de chegada.
- Não-preemptivo: não há interrupção; mesmo que outro processo mais curto ou prioritário chegue, ele deve esperar.

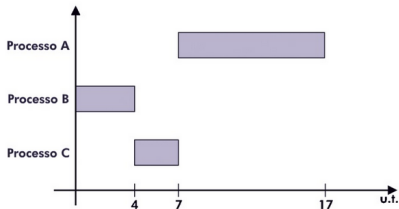


First-Come, First Served (FCFS)

- Execução sequencial: o processo que está no início da fila recebe a CPU e executa até terminar.



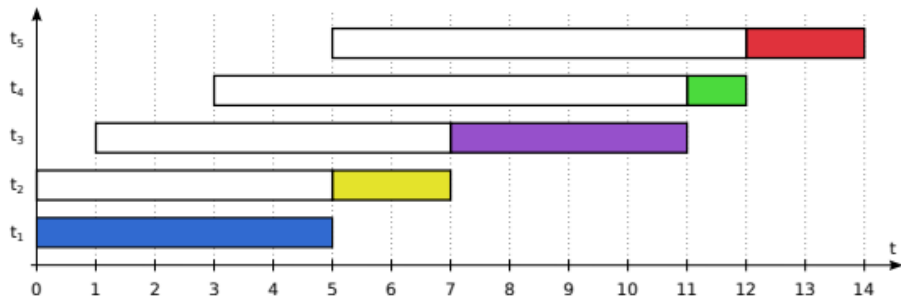
Processo	Tempo de processador (u.t.)
A	10
B	4
C	3



- Se o primeiro for muito lento, todos os outros ficam parados atrás dele (Efeito Comboio)



First-Come, First Served (FCFS)



$$\begin{aligned}T_t &= \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{(5-0) + (7-0) + (11-1) + (12-3) + (14-5)}{5} \\&= \frac{5 + 7 + 10 + 9 + 9}{5} = \frac{40}{5} = 8,0s\end{aligned}$$

$$\begin{aligned}T_w &= \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{(0-0) + (5-0) + (7-1) + (11-3) + (12-5)}{5} \\&= \frac{0 + 5 + 6 + 8 + 7}{5} = \frac{26}{5} = 5,2s\end{aligned}$$



Round-Robin (RR)

- Cada processo ganha um "fatia de tempo"(quantum).
 - Ou seja é preemptivo
- Quando o tempo acaba, ele vai para o fim da fila.
- É o que o Linux e o Windows usam na base para dar a sensação de multitarefa.

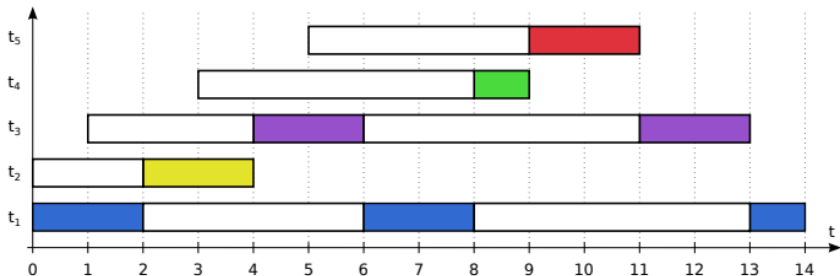


- Justiça: todos os processos recebem tempo de CPU de forma igualitária.
- Ideal para sistemas interativos, pois garante que processos de usuário não fiquem bloqueados por muito tempo.
- **Analogia:**
 - Rodízio de fala em sala



Round-Robin (RR)

- Quando o tempo acaba, a tarefa vai para o fim da fila



- Nenhum processo fica sem executar, mesmo que outros sejam longos.

$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{14 + 4 + 12 + 6 + 6}{5} = \frac{42}{5} = 8,4s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{9 + 2 + 8 + 5 + 4}{5} = \frac{28}{5} = 5,6s$$

- Quantum muito grande
 - RR se aproxima do FCFS
- Quantum muito pequeno
 - excesso de context switching (trocas de processo), o que reduz a eficiência.



Round-Robin (RR)

- Quando o tempo acaba, a tarefa vai para o fim da fila

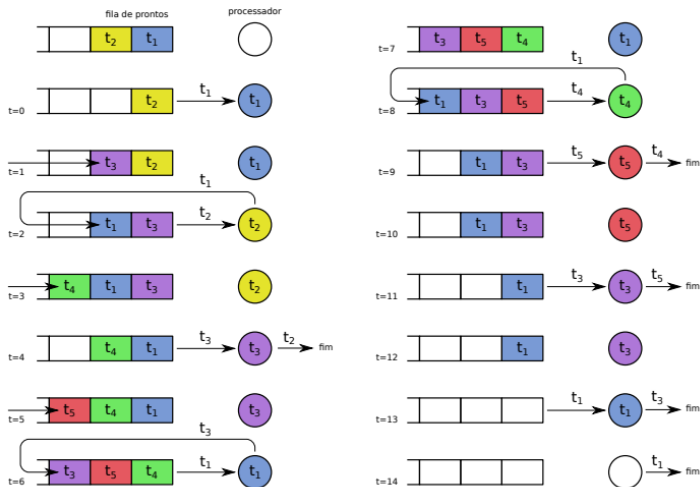


Tabela: Vantagens e Desvantagens do Round-Robin

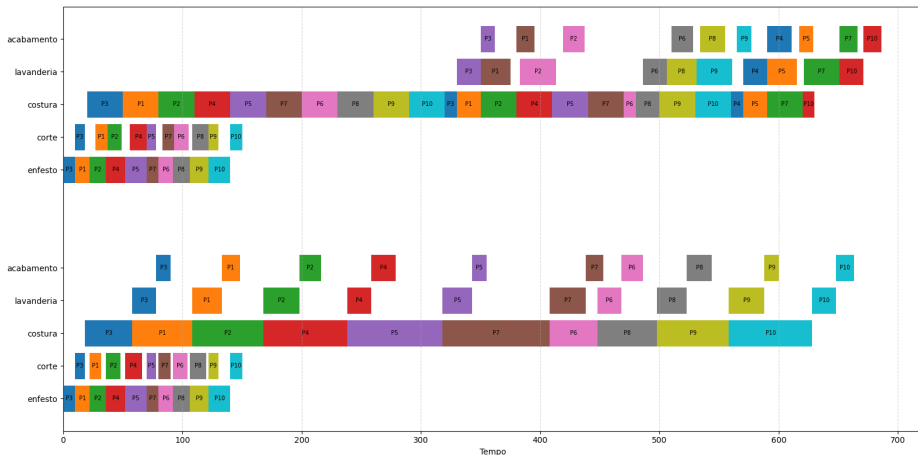
Vantagens	Desvantagens
Justo e simples	Muitas trocas de contexto se quantum for pequeno
Evita inanição	Tempo médio de espera pode aumentar
Bom para sistemas interativos	Desempenho depende da escolha do quantum

Tabela: Comparação rápida: FCFS vs Round-Robin

Critério	FCFS	Round-Robin (RR)
Tipo	Não-preemptivo	Preemptivo
Ordem	Ordem de chegada	Ordem circular
Justiça	Pode favorecer processos longos	Todos recebem tempo igual
Responsividade	Baixa	Alta
Problema principal	Convoy effect	Excesso de trocas de contexto

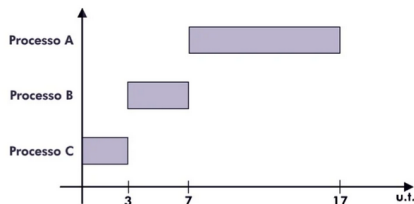


Comparativo em outro cenário



Shortest Job First (SJF)

- A tarefa mais curta passa na frente.
- Ótimo para vazão, mas tarefas longas podem nunca rodar.
- **Não-preemptivo:** o processo com menor tempo de CPU é escolhido e executado até terminar.
- **Objetivo:** minimizar o tempo médio de espera e de retorno, tornando o escalonamento mais eficiente.

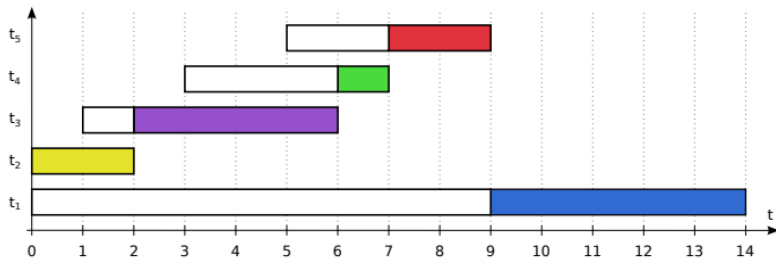


- **Vantagem:**
 - Menor tempo médio
- **Problema:**
 - Difícil prever duração
 - Pode causar starvation



Shortest Job First (SJF)

- Justiça relativa: favorece processos curtos, mas pode prejudicar processos longos.



$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{14 + 2 + 5 + 4 + 4}{5} = \frac{29}{5} = 5,8s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{9 + 0 + 1 + 3 + 2}{5} = \frac{15}{5} = 3,0s$$

- Eficiência: reduz o tempo médio de espera em comparação ao FCFS.
- Necessidade de previsão: exige conhecer ou estimar o tempo de execução dos processos, o que nem sempre é trivial.



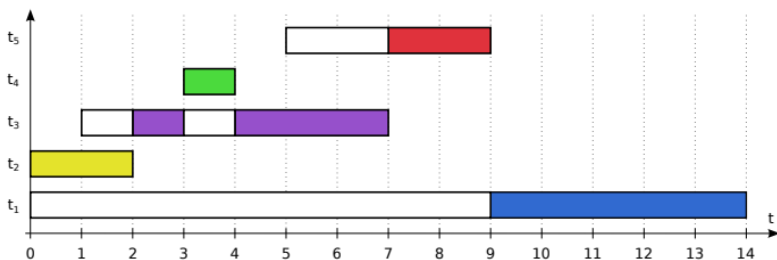
Shortest Remaining Time First (SRTF)

- Versão preemptiva do Shortest Job First (SJF)
- **Ideia:**
 - Interrompe se surgir processo menor
- Fila dinâmica: a cada chegada de processo, o escalonador compara o tempo restante de execução.
- Preempção: se o novo processo tiver tempo menor, o processo atual é interrompido e colocado de volta na fila.
- Execução: o processo com menor tempo restante sempre é escolhido.
- **Resultado:**
 - Melhor desempenho médio



Shortest Remaining Time First (SRTF)

- Eficiência: minimiza o tempo médio de espera e de retorno, sendo considerado ótimo nesse aspecto.



$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{14 + 2 + 6 + 1 + 4}{5} = \frac{27}{5} = 5,4s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{9 + 0 + 2 + 0 + 2}{5} = \frac{13}{5} = 2,6s$$

- Preemptivo: garante que processos curtos não fiquem esperando muito tempo.



Shortest Remaining Time First (SRTF)

Tabela: Vantagens e Desvantagens do SRTF

Vantagens	Desvantagens
Tempo médio de espera mínimo	Processos longos podem sofrer inanição
Melhor responsividade para processos curtos	Complexidade maior na implementação
Ótimo em eficiência teórica	Requer conhecimento/estimativa precisa dos burst t

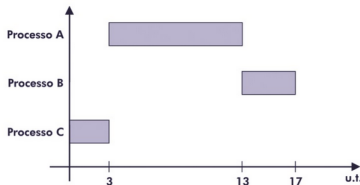
Tabela: Comparação rápida: SJF vs SRTF

Critério	SJF (não-preemptivo)	SRTF (preemptivo)
Tipo	Não-preemptivo	Preemptivo
Escolha	Menor burst time inicial	Menor tempo restante
Responsividade	Boa para processos curtos	Excelente para processos curtos
Problema principal	Inanição de processos longos	Inanição ainda mais acentuada



Escalonamento por Prioridades

- Cada processo recebe um valor de prioridade (pode ser definido pelo sistema ou pelo usuário).
- Maior prioridade → executa primeiro
- Pode ser:
 - Cooperativo: se um processo de maior prioridade chega enquanto outro está em execução, ele só será atendido depois que o atual terminar.
 - Preemptivo: se um processo de maior prioridade chega, o processo atual é interrompido e a CPU é atribuída ao novo processo.



Processo	Tempo de processador (u.t.)	Prioridade
A	10	2
B	4	1
C	3	3

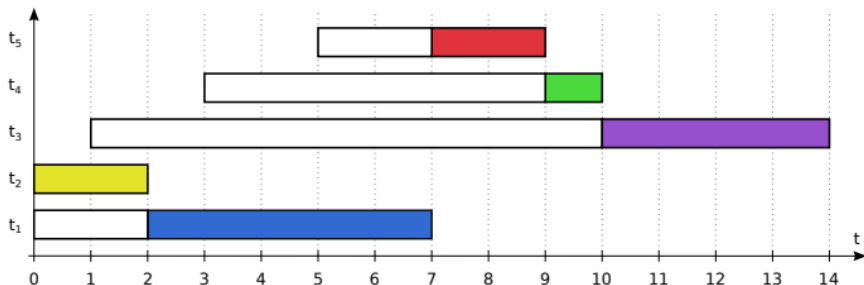
Problema:

- Starvation (Veremos daqui a pouco)



Escalonamento por prioridades fixas - PRIOc

- PRIOc: por prioridades, cooperativo
- Prioridades: $t_1 : 2 \quad t_2 : 3 \quad t_3 : 1 \quad t_4 : 4 \quad t_5 : 5$



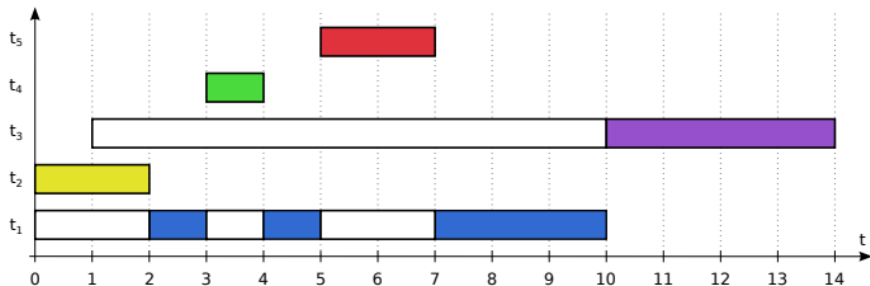
$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{7 + 2 + 13 + 7 + 4}{5} = \frac{33}{5} = 6,6s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{2 + 0 + 9 + 6 + 2}{5} = \frac{19}{5} = 3,8s$$



Escalonamento por prioridades fixas - PRIOp

- PRIOp: por prioridades, preemptivo
- Prioridades: $t_1 : 2$ $t_2 : 3$ $t_3 : 1$ $t_4 : 4$ $t_5 : 5$



$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{10 + 2 + 13 + 1 + 2}{5} = \frac{28}{5} = 5,6s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{5 + 0 + 9 + 0 + 0}{5} = \frac{14}{5} = 2,8s$$



Escalonamento por prioridades fixas (PRIOc, PRIOp)

- Nota-se que

Tabela: Comparação rápida: PRIOc vs PRIOp

Critério	PRIOc (não-preemptivo)	PRIOp (preemptivo)
Interrupção	Não há	Sim, quando chega processo mais priorit
Responsividade	Menor	Maior
Justiça	Mais previsível	Favorece processos críticos
Risco de inanição	Menor	Maior
Complexidade	Mais simples	Mais complexo
Uso típico	Sistemas batch, tarefas longas	Sistemas de tempo real, interativos



Escalonamento por prioridades dinâmicas (PRIOd)

- **Prioridade estática:** se processos de alta prioridade chegarem constantemente, um processo de baixa prioridade pode nunca chegar a executar.
 - Isso é o que chamamos de **Starvation**.

Solução: uso de **prioridades dinâmicas**

- Utilizamos Prioridades Dinâmicas para evitar a Starvation e equilibrar o uso de recursos entre processos de CPU e de I/O.
- Aumentar aos poucos a prioridade das tarefas paradas.
- A tarefa não mantém a mesma prioridade do início ao fim, o SO ajusta sua prioridade em tempo real.
- Ao executar, a tarefa volta à sua prioridade original.

Analogia:

- Cliente esperando muito tempo vira prioridade



■ Ideia:

- Aumentar prioridade com o tempo

■ Objetivo:

- Evitar starvation: um processo fica esperando por tempo indefinido (ou muito longo) e praticamente nunca executa

■ Analogia:

- Fila de restaurante
- Chegam clientes VIP o tempo todo
- Você está na fila comum
- Sempre que sua vez chega, aparece alguém mais importante
- Resultado: você nunca é atendido → starvation



Definições:

N : número de tarefas no sistema

t_i : tarefa i , $1 \leq i \leq N$

pe_i : prioridade estática da tarefa t_i

pd_i : prioridade dinâmica da tarefa t_i

Quando uma tarefa nova t_{nova} ingressa no sistema:

$pe_{nova} \leftarrow \text{prioridade fixa}$

$pd_{nova} \leftarrow pe_{nova}$

Para escolher t_{prox} , a próxima tarefa a executar:

escolher $t_{prox} \mid pd_{prox} = \max_{i=1}^N (pd_i)$

$\forall t_i \neq t_{prox} : pd_i \leftarrow pd_i + \alpha$

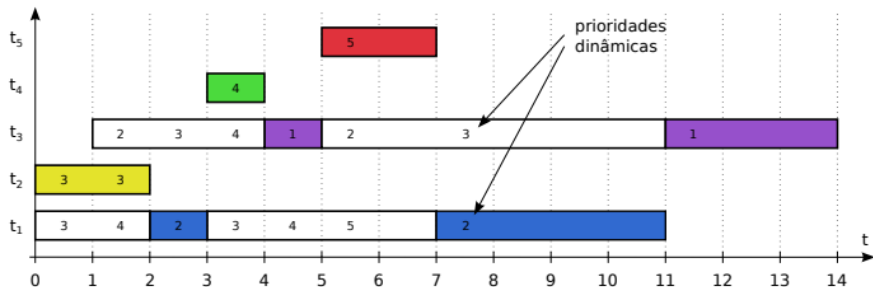
$pd_{prox} \leftarrow pe_{prox}$

Esse mecanismo de ajuste gradual é denominado **Aging** (Envelhecimento).



Escalonamento por prioridades dinâmicas (PRIOd)

- PRIOd: por prioridades, preemptivo e dinâmico
- Prioridades: $t_1 : 2$ $t_2 : 3$ $t_3 : 1$ $t_4 : 4$ $t_5 : 5$



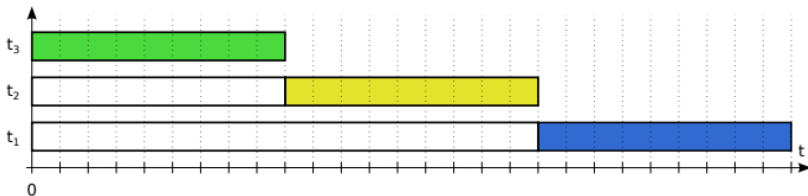
$$T_t = \frac{t_t(t_1) + \dots + t_t(t_5)}{5} = \frac{11 + 2 + 13 + 1 + 2}{5} = \frac{29}{5} = 5,8s$$

$$T_w = \frac{t_w(t_1) + \dots + t_w(t_5)}{5} = \frac{6 + 0 + 9 + 0 + 0}{5} = \frac{15}{5} = 3,0s$$

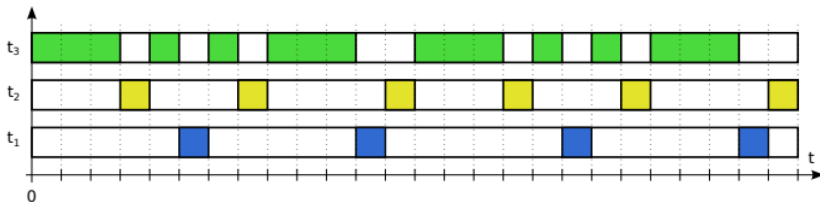


Efeito da prioridade dinâmica

- Round-Robin ($t_q = 1$) e prioridades $t_1 : 1$ $t_2 : 3$ $t_3 : 6$
- Sem envelhecimento:
 - a cada fim de quantum, sempre a tarefa mais prioritária é escolhida



- Com envelhecimento:
 - as três tarefas recebem o processador periodicamente



A prioridade de uma tarefa é determinada por dois grupos de fatores:

■ Fatores Externos (Prioridade Estática):

- Informações providas pelo usuário ou administrador.
- **Exemplos:** Classe do usuário (Diretor vs. Estagiário), valor pago pelo serviço (Premium vs. Básico), importância da tarefa.

■ Fatores Internos (Prioridade Dinâmica):

- Estimados pelo próprio escalonador com base em dados do sistema.
- **Exemplos:** Idade da tarefa, interatividade, uso de memória.
- **Aging (Envelhecimento):** Mecanismo de ajuste para evitar a inanição de processos antigos.



Comparação entre os algoritmos apresentados

Algoritmo de escalonamento	FCFS	RR	SJF	SRTF	PRIOc	PRIOp	PRIOd
Tempo médio de execução T_t	8,0	8,4	5,8	5,4	6,6	5,6	5,8
Tempo médio de espera T_w	5,2	5,6	3,0	2,6	3,8	2,8	3,0
Número de trocas de contexto	4	7	4	5	4	6	6
Tempo total de processamento	14	14	14	14	14	14	14



No Windows, as threads assumem valores entre **0** e **31**, dependendo da classe do processo e da thread.

Classes Padrão:

- 24: Tempo Real
- 13: Alta
- 10: Acima do normal
- 8: **Normal**
- 4: Baixa / Ociosa

Incremento de Janela

A tarefa responsável pela **janela ativa** recebe um incremento interno de **+1** ou **+2** para melhorar a responsividade.



O Linux trabalha com duas escalas distintas e independentes:

■ Tarefas de Tempo Real:

- Escala de **1 a 99** (positiva).
- Maiores valores = Maior prioridade.
- Restrito ao núcleo ou usuário *root*.

■ Demais Tarefas (Escala Nice):

- Padrão UNIX-like: **-20 a +19** (negativa).
- **Atenção:** Maiores valores indicam **menor** prioridade.
- Ajustáveis via comandos *nice* e *renice*.



- SOs modernos (Windows, Linux, MacOS) lidam com tarefas heterogêneas:
 - Jogos, editores de texto, backups, serviços de rede.
- **Estratégia:** Combinação de múltiplas políticas de escalonamento.
- **Estrutura no Linux:** *Multiple Feedback Queues* (Múltiplas Filas com Retroalimentação).
- As tarefas são divididas em **Classes de Escalonamento** com prioridades distintas.

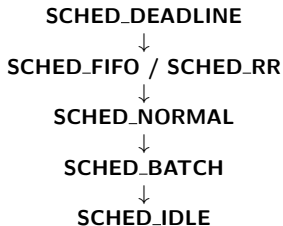


Usa várias filas com políticas e algoritmos distintos:

- **SCHED_DEADLINE**: tarefas de tempo real com prazos, usa o algoritmo Earliest Deadline First (EDF).
- **SCHED_FIFO**: prioridades fixas preemptivo, sem quantum.
- **SCHED_RR**: SCHED_FIFO + Round-Robin, quantum 100 ms.
- **SCHED_NORMAL**: algoritmo CFS - Completely Fair Scheduler (Round-Robin c/ quantum variável, prioridades dinâmicas).
- **SCHED_BATCH**: tarefas CPU-bound de baixa prioridade.
- **SCHED_IDLE**: só recebe CPU se não houver tarefa ativa.



As tarefas são ativadas seguindo uma ordem de precedência rigorosa:



Nota: Classes interativas só executam se não houver tarefas de Tempo Real ativas.



O Algoritmo CFS (Completely Fair Scheduler)

- Utilizado nas classes NORMAL, BATCH e IDLE.
- Baseado no conceito de "escalonamento justo".
- Utiliza a escala de prioridades **nice** (-20 a +19).
- **Objetivo:** Garantir que cada processo receba uma parcela justa do tempo de processador, proporcional ao seu peso (*nice level*).



Escalonadores reais

Classe	DEADLINE	FIFO	RR	NORMAL	BATCH	IDLE
Precedência	1	2	2	3	4	5
Algoritmo	EDF	PRIOp	PRIOp + Round Robin	CFS	CFS	CFS
Tempo real	✓	✓	✓			
Preempção por prioridade	✓	✓	✓	✓	✓	✓
Preempção por tempo			✓	✓	✓	✓
Quantum			100 <i>ms</i>	0,75 a 6 <i>ms</i>	1500 <i>ms</i>	1500 <i>ms</i>
Prioridade fixa	0	1 a 99	1 a 99	0	0	0
Níveis "nice"				-20 a +19	-20 a +19	> +19
Envelhecimento				✓	✓	



Prática



- Observar o comportamento das classes **SCHED_FIFO**, **SCHED_RR** e **SCHED_NORMAL**.
- Manipular prioridades e políticas via comandos de Shell (`chrt`, `nice`).
- Analisar a distribuição de CPU em um único núcleo (`taskset`).
- Entender o conceito de *Starvation* e *Quantum* na prática.



Comando chrt (Change Real-Time)

O comando **chrt** é utilizado para manipular as **Classes de Escalonamento** e as prioridades de Tempo Real (RT).

- **Função:** Permite alternar entre FIFO, RR, BATCH e OTHER.
- **Prioridade RT:** Escala de 1 a 99.

Principais Flags

- **-f:** Define a classe como SCHED_FIFO.
- **-r:** Define a classe como SCHED_RR (Round Robin).
- **-p:** Define a prioridade (deve ser o último argumento antes do PID).

Exemplo Prático

```
$ sudo chrt -f -p 50 1234
```

(Transforma o processo 1234 em Tempo Real FIFO com prioridade 50)



Comando taskset (Afinidade de CPU)

O comando **taskset** permite definir ou recuperar a afinidade de CPU de um processo, determinando em quais núcleos ele pode executar.

- **Conceito:** Força um processo a rodar em um subconjunto específico de CPUs.
- **Importância:** Crucial para observar preempção e disputa de recursos em sistemas *multicore*.
- **Performance:** Evita *cache misses* causados pela migração do processo entre núcleos.

Exemplos de Uso

- Lançar programa restrito ao **Núcleo 0**:

```
$ taskset -c 0 ./meu_programa
```
- Alterar processo (PID 1234) para os **Núcleos 1 e 2**:

```
$ taskset -cp 1,2 1234
```
- Verificar afinidade atual de um processo:

```
$ taskset -p 1234
```



- Este código realiza cálculos intensos para manter a thread em estado *Running*.

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 int main() {
6     printf("PID: %d em execucao...\n", getpid());
7     double x = 1.5;
8     while(1) {
9         x = sqrt(x + rand()); // Loop infinito de calculo
10    }
11    return 0;
12 }
```

- **Compilação:** gcc cpu_stress.c -o cpu_stress -lm
- O -lm é uma instrução para incluir a Biblioteca Matemática padrão do C (libm).

